

Program to demonstrate Simple Inheritance

```
// Create a superclass.

class A {

    int i, j;

    void showij() {

        System.out.println("i and j: " + i + " " + j);

    }

}

// Create a subclass by extending class A.

class B extends A {

    int k;

    void showk() {

        System.out.println("k: " + k);

    }

    void sum() {

        System.out.println("i+j+k: " + (i+j+k));

    }

}

public class SimpleInheritance {

    public static void main(String[] args) {

        A superOb = new A();

        B subOb = new B();

        // The superclass may be used by itself.

        superOb.i = 10;

        superOb.j = 20;

        System.out.println("Contents of superOb: ");

        superOb.showij();

        System.out.println();

        /* The subclass has access to all public members of

        its superclass. */

        subOb.i = 7;
```

```

    subOb.j = 8;

    subOb.k = 9;

    System.out.println("Contents of subOb: ");

    subOb.showij();

    subOb.showk();

    System.out.println();

    System.out.println("Sum of i, j and k in subOb:");

    subOb.sum();

}

}

```

Program to Illustrate Member Access in Inheritance

// Create a superclass.

```

class A {

    int i; // default access

    private int j; // private to A

    void setij(int x, int y) {

        i = x;

        j = y;

    }

}

```

// Sub class.

```

class B extends A {

    int total;

    void sum() {

        total = i + j;

    }

}

public class Access {

    public static void main(String[] args) {

        B subOb = new B();
    }

}

```

```
subOb.setij(10, 12);  
subOb.sum();  
System.out.println("Total is " + subOb.total);  
}  
}
```

Program to Demonstrate when constructors are executed.

```
// Create a super class.  
class A {  
    A() {  
        System.out.println("Inside A's constructor.");  
    }  
}  
  
// Create a subclass by extending class A.  
class B extends A {  
    B() {  
        System.out.println("Inside B's constructor.");  
    }  
}  
  
// Create another subclass by extending B.  
class C extends B {  
    C() {  
        System.out.println("Inside C's constructor.");  
    }  
}  
  
class CallingCons {  
    public static void main(String[] args) {  
        C c = new C();  
    }  
  
    // Using super to overcome name hiding.
```

```

class A {
    int i;
}

// Create a subclass by extending class A.
class B extends A {
    int i; // this i hides the i in A

    B(int a, int b) {
        super.i = a; // i in A
        i = b; // i in B
    }

    void show() {
        System.out.println("i in superclass: " + super.i);
        System.out.println("i in subclass: " + i);
    }
}

class UseSuper {
    public static void main(String[] args) {
        B subOb = new B(1, 2);
        subOb.show();
    }
}

```

Program to Illustrate Method overriding.

```

class A {
    int i, j;

    A(int a, int b) {
        i = a;
        j = b;
    }

    // display i and j
    void show() {
        System.out.println("i and j: " + i + " " + j);
    }
}

```

```
}  
}  
class B extends A {  
    int k;  
    B(int a, int b, int c) {  
        super(a, b);  
        k = c;  
    }  
    // display k – this overrides show() in A  
    void show() {  
        System.out.println("k: " + k);  
    }  
}  
class Override {  
    public static void main(String[] args) {  
        B subOb = new B(1, 2, 3);  
        subOb.show(); // this calls show() in B  
    }  
}
```